第十章 索引与散列详细举例详解

一、索引基本概念详细举例

1.1 索引实际类比

图书馆示例：

- 没有索引：找一本书需要遍历整个图书馆，可能需要检查10万本书
- 有作者索引：先查作者目录（可能在几页纸内），找到索书号，直接到对应书架
- 有书名索引：先查书名目录，再到对应位置

数据库类比：

假设有100万条学生记录的表：

```
-- 没有索引的查询（全表扫描）
SELECT * FROM students WHERE student_id = 123456;
-- 需要检查100万行记录

-- 有索引的查询
CREATE INDEX idx_student_id ON students(student_id);
SELECT * FROM students WHERE student_id = 123456;
-- 索引可能只需检查3-4层B+树（约3-4次磁盘I/O）
```

二、有序索引详细例子

2.1 聚集索引具体例子

假设有一个 `Employee`表，按 `employee_id`聚集存储：

```
物理存储顺序（磁盘块1-4）：
块1: [101, "John", 50000]
块2: [102, "Alice", 55000]
块3: [103, "Bob", 48000]
块4: [104, "Carol", 52000]

聚集索引结构：
搜索键     指针
101    → 块1, 位置1
102    → 块2, 位置1
103    → 块3, 位置1
104    → 块4, 位置1
```

查询示例：

```
-- 范围查询高效
SELECT * FROM Employee
WHERE employee_id BETWEEN 102 AND 104;
-- 可以利用聚集索引顺序读取连续的磁盘块
```

2.2 稠密索引详细例子

假设 `Employee`表有5条记录：

```
数据文件：
记录1: [101, "John", "Sales", 50000]
记录2: [102, "Alice", "IT", 55000]
记录3: [103, "Bob", "HR", 48000]
记录4: [104, "Carol", "Sales", 52000]
记录5: [105, "David", "IT", 53000]

稠密索引文件（按employee_id排序）：
搜索键     指针
101    → 记录1地址
102    → 记录2地址
```

```
103    → 记录3地址
104    → 记录4地址
105    → 记录5地址
```

查询过程 ：

```
查找employee_id=103的记录：
1. 在索引文件二分查找103
2. 找到索引条目[103 → 记录3地址]
3. 直接访问记录3地址
4. 总共：1次索引I/O + 1次数据I/O
```

2.3 稀疏索引详细例子

假设每个磁盘块可存放2条记录：

```
数据文件（3个磁盘块）：
块1: [101, "John"], [102, "Alice"]
块2: [103, "Bob"], [104, "Carol"]
块3: [105, "David"], [106, "Eve"]

稀疏索引文件：
搜索键     指针
101    → 块1起始地址
103    → 块2起始地址
105    → 块3起始地址
```

查询过程 ：

```
查找employee_id=104的记录：
1. 在索引中找≤104的最大键 → 103
2. 根据指针访问块2
3. 在块2内顺序查找
4. 找到记录[104, "Carol"]
5. 总共：1次索引I/O + 1次数据I/O + 块内搜索
```

三、B+树详细示例（n=4情况）

3.1 B+树结构示例

假设B+树参数n=4（每个节点最多3个键，4个指针）

完整B+树示例 ：

```
根节点：
[10, 20, 30]
  |    |    |    |
  ↓    ↓    ↓    ↓
[5,8] [15,18] [25,28] [35,38,40]
 叶子节点链表

叶子节点内容（每个条目是[键, 记录指针]）：
叶子1: [5, ptr1] [8, ptr2]      → 下一个叶子2
叶子2: [10, ptr3] [15, ptr4]    → 下一个叶子3
叶子3: [18, ptr5] [20, ptr6]    → 下一个叶子4
叶子4: [25, ptr7] [28, ptr8]    → 下一个叶子5
叶子5: [30, ptr9] [35, ptr10]   → 下一个叶子6
叶子6: [38, ptr11] [40, ptr12] → NULL
```

3.2 B+树插入详细步骤

初始状态（简单B+树，n=3）：

```
根节点（也是叶节点）: [10, 20]
```

步骤1：插入5

```
节点变成: [5, 10, 20]   （未满，直接插入）
```

步骤2：插入15

节点变成：[5, 10, 15, 20] （已满，n=3最多3个键）

需要分裂：
1. 分成两个节点：[5, 10] 和 [15, 20]
2. 创建新的根节点：[15]
3. 根节点指向两个叶节点

结果：
```
       [15]
      /    \
[5,10]    [15,20]
```

步骤3：插入25

1. 找到应插入的叶节点[15,20]
2. 插入后变成[15,20,25]（未满，n=3最多3个键）

结果：
```
       [15]
      /    \
[5,10]    [15,20,25]
```

步骤4：插入30（触发分裂）

1. 叶节点[15,20,25]已满
2. 分裂为[15,20]和[25,30]
3. 向上插入25到父节点[15]，变成[15,25]
4. 父节点未满（n=3最多3个键）

最终结果：
```
       [15,25]
      /   |   \
[5,10] [15,20] [25,30]
```

3.3 B+树删除详细步骤

初始状态 ：

```
[15,25]
      /   |   \
[5,10] [15,20] [25,30,35]
```

步骤1：删除30

叶节点[25,30,35]变成[25,35]（键数≥⌈(n-1)/2⌉=⌈(3-1)/2⌉=1，满足）

结果：
```
       [15,25]
      /   |   \
[5,10] [15,20] [25,35]
```

步骤2：删除35（触发合并）

1. 叶节点[25,35]删除35后变成[25]
2. 键数=1，等于最小值1，需要检查兄弟节点
3. 左兄弟[15,20]有2个键，可以重新分配
4. 从左兄弟借一个键20过来
5. 更新父节点键值

结果：
```
       [15,20]
      /   |   \
[5,10] [15] [20,25]
```

详细过程：
- 左兄弟[15,20]借出最大键20
- 当前节点变成[20,25]
- 父节点中25变成20

步骤3：删除15（从[5,10,15]叶节点）

1. 叶节点[5,10,15]删除15变成[5,10]
2. 键数=2 ≥ 最小值1，无需调整

最终结果：

```
       [20]
      /    \
[5,10]    [20,25]
```

四、哈希索引详细示例

4.1 静态哈希示例

假设哈希函数：h(K) = K mod 5

桶数：5

每条记录包含键K

插入记录 ：

插入键值：3, 8, 13, 18, 23, 28

哈希计算：
h(3)=3, h(8)=3, h(13)=3, h(18)=3, h(23)=3, h(28)=3

结果（假设每个桶只能放2条记录）：
桶0: 空
桶1: 空
桶2: 空
桶3: [3] → [8] → [13] → [18] → [23] → [28]（溢出链很长）
桶4: 空

问题 ：哈希冲突严重，查询效率下降

4.2 可扩展哈希详细示例

初始：全局深度=1，局部深度=1，2个桶（0,1）

插入记录（键的二进制表示）：

记录键：2(10), 3(11), 5(101), 7(111)
哈希函数：取最后1位（因为全局深度=1）

步骤1：插入2(10)

h(10)=0 → 桶0
桶0: [2]
桶1: 空
目录：00→桶0, 01→桶1

步骤2：插入3(11)

h(11)=1 → 桶1
桶0: [2]
桶1: [3]
目录：00→桶0, 01→桶1

步骤3：插入5(101)（假设每个桶容量=1）

h(101)=1 → 桶1已满，需要分裂
1. 增加局部深度为2
2. 分裂桶1为桶1和桶3
3. 重新哈希：3(011)留在桶1，5(101)到桶3
4. 更新目录：01→桶1, 11→桶3
5. 全局深度仍为1，但目录项指向不同桶

结果：
目录：00→桶0, 01→桶1, 10→桶0, 11→桶3
桶0: [2]
桶1: [3]
桶3: [5]

步骤4：插入7(111)

```
h(111)=1 → 目录项11指向桶3
桶3已有[5]，未满，直接插入
桶3: [5,7]
```

五、位图索引详细示例

5.1 学生表位图索引

假设 Students表：

ID	姓名	性别	年级	专业
1	张三	男	大一	计算机
2	李四	女	大二	数学
3	王五	男	大一	物理
4	赵六	女	大三	计算机
5	钱七	男	大二	数学

性别位图 ：

```
男: 1 0 1 0 1  (第1,3,5行为男)
女: 0 1 0 1 0  (第2,4行为女)
```

年级位图 ：

```
大一: 1 0 1 0 0
大二: 0 1 0 0 1
大三: 0 0 0 1 0
```

专业位图 ：

```
计算机: 1 0 0 1 0
数学:   0 1 0 0 1
物理:   0 0 1 0 0
```

5.2 复杂查询示例

查询1 ：大一的男生

```
大一: 1 0 1 0 0
AND
男:   1 0 1 0 1
结果: 1 0 1 0 0 → 第1,3行（张三、王五）
```

查询2 ：计算机专业或大二的学生

```
计算机: 1 0 0 1 0
OR
大二:   0 1 0 0 1
结果: 1 1 0 1 1 → 第1,2,4,5行
```

查询3 ：不是数学专业的大三学生

```
数学:   0 1 0 0 1
NOT数学:1 0 1 1 0
AND
大三:   0 0 0 1 0
结果: 0 0 0 1 0 → 第4行（赵六）
```

六、复合索引详细示例

6.1 复合索引排序示例

在 (dept_name, salary)上创建复合索引：

假设 Employee表数据：

```
[101, "Sales", 50000]
[102, "Sales", 55000]
[103, "IT", 60000]
[104, "IT", 58000]
[105, "HR", 45000]
[106, "HR", 48000]
```

复合索引按字典序排序：

```
键值                    记录指针
("HR", 45000)       → 记录5
("HR", 48000)       → 记录6
("IT", 58000)       → 记录4
("IT", 60000)       → 记录3
("Sales", 50000)    → 记录1
("Sales", 55000)    → 记录2
```

6.2 复合索引查询示例

```
-- 情况1：等值查询（高效）
SELECT * FROM Employee
WHERE dept_name = 'IT' AND salary = 60000;
-- 可以直接在索引中找到("IT", 60000)

-- 情况2：前缀查询（高效）
SELECT * FROM Employee
WHERE dept_name = 'IT';
-- 可以找到索引中所有"IT"开头的条目

-- 情况3：非前缀查询（低效）
SELECT * FROM Employee
WHERE salary = 60000;
-- 无法使用索引的有序性，需要扫描整个索引
-- 因为索引按(dept_name, salary)排序，不是按salary排序
```

七、真题应用题示例

7.1 B+树参数计算（填空题）

题目 ：一个B+树的节点大小为4KB，键大小8B，指针大小8B，求：

1. 每个节点最多可容纳的键数n
2. 非叶节点最少子节点数
3. 叶节点最少键数

计算过程 ：

```
1. 设每个节点有K个键，K+1个指针
   总大小 = 8K + 8(K+1) = 16K + 8 ≤ 4096
   16K ≤ 4088
   K ≤ 255.5
   取K=255，所以n=256（指针数）

2. 非叶节点最少子节点数 = ⌈n/2⌉ = ⌈256/2⌉ = 128

3. 叶节点最少键数 = ⌈(n-1)/2⌉ = ⌈255/2⌉ = 128
```

7.2 索引选择优化（综合题）

场景 ：学生选课系统

```
-- 表结构
Students(sid, sname, gender, dept)
Courses(cid, cname, credits)
Enrollments(sid, cid, grade, semester)
```

```
-- 查询1：查询计算机系学生的选课情况
SELECT s.sname, e.cid, e.grade
FROM Students s JOIN Enrollments e ON s.sid = e.sid
WHERE s.dept = 'CS';

-- 查询2：查询2023秋季学期数据库课程的成绩
SELECT s.sname, e.grade
FROM Students s
JOIN Enrollments e ON s.sid = e.sid
JOIN Courses c ON e.cid = c.cid
WHERE c.cname = 'Database' AND e.semester = 'Fall2023';
```

索引设计建议 ：

1. Students表：在dept上创建索引（支持查询1）
2. Enrollments表：创建复合索引(sid, cid)（支持连接）
3. Enrollments表：创建复合索引(cid, semester)（支持查询2）
4. Courses表：在cname上创建索引

I/O代价分析 ：

无索引：
查询1需要扫描整个Students表（假设10000行）和Enrollments表（假设50000行）

有索引：
查询1可以使用dept索引找到CS系学生（假设1000人）
再通过sid索引连接，大幅减少I/O

八、索引创建实战示例

8.1 各种索引创建语句

```
-- 1. 创建主键索引（自动创建）
CREATE TABLE Students (
    sid INT PRIMARY KEY,  -- 自动创建聚集索引
    sname VARCHAR(50)
);

-- 2. 创建唯一索引
CREATE UNIQUE INDEX idx_sname ON Students(sname);

-- 3. 创建复合索引
CREATE INDEX idx_dept_gender ON Students(dept, gender);

-- 4. 创建覆盖索引
CREATE INDEX idx_dept_covering ON Students(dept) INCLUDE (sname, email);
-- 查询时只需访问索引，无需访问表
SELECT sname, email FROM Students WHERE dept = 'CS';

-- 5. 创建函数索引
CREATE INDEX idx_lower_sname ON Students(LOWER(sname));
-- 支持大小写不敏感查询
SELECT * FROM Students WHERE LOWER(sname) = 'zhangsan';
```

8.2 索引使用情况分析

```
-- 查看索引使用情况
EXPLAIN SELECT * FROM Students WHERE dept = 'CS';

-- 强制使用特定索引
SELECT * FROM Students WITH(INDEX(idx_dept_gender))
WHERE dept = 'CS';

-- 重建索引（整理碎片）
ALTER INDEX idx_dept_gender ON Students REBUILD;
```

九、常见错误示例

9.1 无效索引使用

```
-- 错误1：在索引列上使用函数
CREATE INDEX idx_sname ON Students(sname);
SELECT * FROM Students WHERE UPPER(sname) = 'ZHANGSAN';
-- 无法使用索引，因为对列使用了函数

-- 错误2：前导列未在查询条件中
CREATE INDEX idx_dept_gender_age ON Students(dept, gender, age);
SELECT * FROM Students WHERE gender = 'M' AND age > 20;
-- 无法有效使用复合索引，因为dept未在条件中

-- 错误3：对高选择性差的列建索引
CREATE INDEX idx_gender ON Students(gender);
-- 性别只有2-3个值，索引效果差
```

9.2 索引维护问题

```
-- 频繁更新的列建索引需谨慎
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    status VARCHAR(20),  -- 频繁更新
    INDEX idx_status(status)  -- 每次更新status都需要更新索引
);

-- 解决方案：考虑是否需要此索引
-- 如果查询不多，可能不需要
-- 或者使用位图索引（如果数据库支持）
```

这些详细示例覆盖了索引的主要概念和实际应用场景。如果还有不够具体的地方，请告诉我具体哪个知识点需要更详细的例子。